SoulCare Stress Management Web Application(Group A)

A PROJECT REPORT SUBMITTED BY

R.M.G.R.Rathnayaka

(S / 20 / 139)

CSC3213 - Project in Computer Science I

Department of Statistics and Computer Science

of the

FACULTY OF SCIENCE

UNIVERSITY OF PERADENIYA

SRI LANKA

2025

ABSTRACT

Mental health is a critical aspect of overall well-being, yet access to quality care and effective management tools remains a challenge for many. To address this, we developed **SoulCare**, a comprehensive web-based platform designed to bridge the gap between patients and mental healthcare providers, including doctors and counselors. The system creates a holistic ecosystem that integrates clinical management functionalities with patient-centric self-care resources.

The platform enables patients to proactively manage their mental health through self-assessment features, such as a specialized stress questionnaire and interactive games designed to analyze cognitive responses and stress levels. It also provides tools for daily mood tracking and journaling to foster self-awareness. For healthcare providers, SoulCare offers a suite of administrative and clinical tools to efficiently manage appointment schedules, issue prescriptions, and monitor patient progress through detailed statistical dashboards.

To facilitate seamless care, the system includes robust communication modules, such as real-time messaging and automated email notifications. Furthermore, it hosts a comprehensive content library and blog section to distribute mental health education. Developed using a **React** frontend and a **Django** backend, SoulCare ensures a secure, responsive, and data-driven environment for all users. This project demonstrates how modern web technologies can be leveraged to enhance the accessibility, privacy, and efficacy of mental healthcare delivery.

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to Dr. Erunika Dayaratne, our course lecturer, for her valuable guidance throughout this project. I am also thankful to our supervisor, Dr. Ruwan Nawarathna, for his expert advice and feedback, which greatly helped in making this project a success.

A special thanks goes to Dr. Kanthi Hettigoda, Lecturer at the Psychology Department, Faculty of Arts, for her guidance in shaping the SoulCare platform to meet professional standards.

Finally, I would like to thank my dedicated team members. Their hard work and cooperation made this project possible.

TABLE OF CONTENT

1. **Introduction**
    1.1 Background & Motivation
    1.2 Project Overview
    1.3 Individual Contributions
2. **Method**
    2.1 Authentication & Security Architecture
    2.2 Administrative Governance & Dashboard
    2.3 Clinical Management Systems
    2.4 Patient Engagement & Self-Care Tools
    2.5 Communication Infrastructure
3. **Implementation and Results**
    3.1 Authentication & Security Infrastructure
    3.2 Administrative Governance Dashboard
    3.3 Clinical Management System
    3.4 Communication & Engagement Modules
4. **Conclusion and Future Work**
    4.1 Conclusion
    4.2 Future Work
5. **References**
6. **Appendices**

1. INTRODUCTION

1.1 Background and Motivation

In recent years, the global conversation around mental health has shifted significantly, with an increasing recognition of its importance to overall well being. However, traditional mental healthcare systems often face challenges related to accessibility, stigma, and the continuity of care. Patients may find it difficult to secure timely appointments, while healthcare providers often lack integrated tools to monitor their patients' progress between sessions. The **SoulCare** project was conceived to address these gaps by leveraging modern web technologies. Our goal was to create a secure, accessible, and holistic platform that connects patients with mental health professionals Doctors and Counselors while also empowering users with self-care tools like mood tracking,Journaling and stress assessment games.

1.2 Project Overview

SoulCare is a robust, full-stack web application designed to facilitate a complete mental healthcare ecosystem. Built with a responsive **React** frontend and a scalable **Django** backend, the system utilizes **MySQL** as its primary relational database management system to ensure secure and efficient data storage. The platform serves four distinct user roles, each equipped with tailored functionalities to ensure seamless care delivery:

- **Patients:** The platform empowers patients to take charge of their mental journey. They can register for an account, securely log in, and access a suite of self assessment tools, including a specialized stress questionnaire and interactive mental games that subtly measure cognitive responses and stress levels. Patients can track their daily emotional state via the mood tracker, maintain a private journal, and access a curated content library for mental health education. Crucially, they can browse verified medical providers(Doctors, Counselors), book appointments, and communicate directly via real time messages.

- **Doctors:** Medical professionals have access to a dedicated dashboard that provides statistical insights into their practice. They can manage their appointment schedules, accept or decline booking requests and issue digital prescriptions directly through the

platform. Doctors can also monitor patient history and progress notes to inform their clinical decisions.Doctors also can share content and blog posts related to mental health.

- **Counselors:** Similar to doctors, counselors utilize a specialized dashboard to manage therapy sessions and track patient well being. They can view patient mood logs (with permission), manage appointments, and contribute therapeutic content and blog posts to the platform's library to aid in patient recovery.

- **Administrators:** Admins play a vital role in maintaining platform integrity and quality assurance. They have a comprehensive dashboard to verify the credentials of new doctors and counselors, ensuring only qualified professionals are listed. Admins also moderate user generated content, manage the blog section, and oversee the Feedback Management System, allowing them to review and approve user testimonials for public display. Additionally, they have the unique ability to export patient game metrics for data analysis.

1.3 Individual Contribution

My individual responsibilities were focused on establishing the system's core architecture, security, administrative governance, and provider patient communication channels. My specific contributions to the project are as follows:

- **Authentication & Security:** I designed and implemented the complete Account Login and Registration systems, incorporating a secure Forgot Password recovery feature via email. This infrastructure utilizes JWT based authentication and enforces strict role based access controls for Patients, Doctors, Counselors, and Admins to ensure data privacy.

- **Administrative Module:** I developed the comprehensive Admin Dashboard and its associated management functionalities. This includes the interfaces and logic for Managing Doctors, Counselors, and Patients, as well as moderating the Content Library and Blog sections. I also implemented the Game Matrix Download feature, enabling admins to export patient game data for analysis and Feedback Management System, allowing review and approval of user testimonials for public display .

- **Provider Tools:** I built the dedicated Doctor and Counselor Dashboards, which provide statistical insights into their practice. I also implemented the Rating System, allowing patients to rate their experience with providers.

- **Clinical Management:** I engineered the Appointment Scheduling System, allowing patients to book Appointments and providers to manage their availability. I also developed the Email Management System to handle notifications for account verification, appointments, newly issued prescriptions and blog posts verification.

- **Communication & Resources:** To facilitate direct interaction, I implemented the Messages system (real time chat). I also built the Content Library, a centralized repository where providers can share therapeutic resources with patients.

2. METHOD

The development of the SoulCare platform followed an agile methodology, characterized by iterative development cycles, continuous testing, and the progressive integration of complex features. My individual responsibilities encompassed the end to end development of the backend architecture using Django Rest Framework (DRF) and the frontend implementation using React with TypeScript. The following sections detail the design and implementation methodologies for each core functionality I contributed.

2.1 Authentication and Security Infrastructure

A secure and scalable authentication system was the foundation of the platform.

- **User Management:** I extended Django's `AbstractUser` model to create a custom `User` model, incorporating a `role` field (Patient, Doctor, Counselor, Admin). This design allowed for a unified login system while maintaining strict separation of privileges.

- **Profile Architecture:** To manage diverse user data, I implemented separate profile models (`DoctorProfile`, `CounselorProfile`, `PatientProfile`) linked via One to One relationships to the user model. This ensured that sensitive medical data (like license numbers) was isolated from general account information.

- **Secure Access:** I implemented JWT (JSON Web Token) authentication using the `SimpleJWT` library. The system issues short lived access tokens for API requests and long lived refresh tokens for session management, enhancing security against session hijacking.

- **Account Recovery:** I developed a secure "Forgot Password" feature. This flow utilizes Django's `PasswordResetView` to generate a cryptographically signed, time sensitive token, which is emailed to the user, allowing them to reset their credentials securely without admin intervention.

2.2 Administrative Dashboard and Governance

The Admin Dashboard was designed as a central command center for platform integrity.

- **User Management:** I developed a reusable table component (`AdminUserTable`) to list users with filtering capabilities. I implemented a verification workflow where admins must review uploaded license documents (stored securely via `FileField` and served as protected media) before toggling a provider's status to "Verified."

- **Content Moderation:** I built a "Feedback Management" module where user testimonials are stored with an `is_approved=False` default state. I created admin only API endpoints (`approve/`, `reject/`) to toggle this state, ensuring only approved content appears on the public landing page.

- **Data Export:** To facilitate analysis of the stress assessment games, I implemented a CSV export feature using Python's `csv` library. This backend view dynamically queries the specific game model (e.g., `ReactionTimeResult`), formats the data, and streams it as a downloadable file response.

2.3 Clinical Management Systems

The core of the clinical management functionality is the Appointment Scheduling System, which I engineered to handle complex booking workflows and state transitions.

- **Appointment Scheduling & Booking Logic:** I designed a robust booking engine that facilitates the interaction between patients and providers.

    - **Conflict Resolution:** The backend logic (implemented in `AppointmentViewSet`) ensures data integrity by validating incoming booking requests against existing records, preventing double booking or invalid time slots.

    - **Data Precision:** I utilized Django's `DateTimeField` in conjunction with custom `save()` method overrides to calculate and store precise appointment

timestamps by combining date and time inputs. This ensures accurate sorting and filtering across different time zones.

- **Lifecycle State Management:** To manage the flow of an appointment from request to completion, I implemented a state machine logic.
    - **Status Transitions:** Appointments transition through defined states: Pending (initial request), Scheduled (provider confirmed), Completed (session finished), and Cancelled.
    - **Custom Actions:** I developed custom API endpoints (e.g., `update_status`, `cancel_by_patient`) using DRF's `@action` decorator. These endpoints encapsulate the business logic for state changes, enforcing strict permission checks ensuring, for example, that only the assigned provider can mark an appointment as "Completed" and only the relevant patient or provider can cancel it.
    - **Audit Trail:** The system tracks metadata such as who initiated a cancellation (patient vs. provider) by updating specific fields (`cancelled_by`), which is then used to generate accurate activity feeds and notifications.

2.4 Patient Engagement and Self-Care Tools

For the patient side, I focused on building tools that encourage daily engagement and self-monitoring.

- **Content Library:** I built a resource sharing system where providers can upload educational materials (PDFs, Videos). I implemented filtering logic to ensure patients can access a public library of resources as well as private content shared specifically with them by their therapists.

- **Rating System:** To foster accountability, I implemented a rating and review system. Patients can rate their experience with providers after appointments. I wrote backend signals to automatically update the provider's average rating score whenever a new review is submitted.

2.5 Communication Infrastructure

To facilitate remote care, I implemented two key communication channels:

- **Real-Time Messaging:** Moving beyond standard HTTP, I implemented a WebSocket-based chat system using Django Channels and Redis. This architecture allows for persistent, real time connections between patients and providers, enabling instant message delivery without page reloads.

- **Email Management System:** I developed a centralized notification service using Django's `core.mail`. This system automatically sends transactional emails for critical events such as account verification, appointment confirmations, prescription issuance, content sharing and blogpost approval.

3. IMPLEMENTATION AND RESULTS

3.1 Authentication and Security Infrastructure

The security and integrity of the SoulCare platform rely on a robust authentication system designed to protect sensitive user data and enforce strict access controls. My implementation focused on creating a scalable, token based authentication mechanism that supports multiple user roles while ensuring a seamless user experience.

Technical Implementation Details:

1. Custom User Model Strategy: Instead of using Django's default user model or creating separate tables for each user type, I extended `AbstractUser`. This approach allowed me to add a critical `role` field directly to the user entity. This design choice simplifies authentication queries checking `request.user.role` is faster and cleaner than querying multiple related tables.

    - Role Separation: By defining distinct choices (`'patient'`, `'doctor'`, `'counselor'`, `'admin'`), the system can enforce permissions at the view level. For instance, a view protected by `IsDoctor` permission simply checks this field, rejecting unauthorized requests instantly.

Code Snippet: Custom User Model,

```
class User(AbstractUser):
    ROLE_CHOICES = [
        ('user', 'User'),
        ('doctor', 'Doctor'),
        ('counselor', 'Counselor'),
        ('admin', 'Admin'),
```

```
    ]
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='user')
    is_verified = models.BooleanField(default=False) # Used for provider verification
```

2. JWT Authentication (JSON Web Tokens): I chose `SimpleJWT` over session based authentication to ensure the backend remains stateless and scalable.

- Access Tokens: Short lived tokens (e.g., 5-15 minutes) are used for API authorization. If stolen, their limited lifespan minimizes risk.

- Refresh Tokens: Long-lived tokens (e.g., 24 hours) allow the frontend to transparently request new access tokens without forcing the user to log in again.

- Frontend Context: I built a React Context (`AuthProvider`) that wraps the application. It initializes by checking for an existing token in local storage. It also sets up an Axios interceptor that automatically attaches the `Authorization: Bearer <token>` header to every outgoing request, ensuring consistent security without repetitive code in every component.

3. Secure Forgot Password Flow: Recovering a lost account is a high-risk operation. I implemented a secure flow leveraging Django's cryptographic utilities to prevent unauthorized account takeovers.

- Token Generation: When a user requests a reset, the backend uses `default_token_generator` to create a token. This token is a hash of the user's ID, password hash, and timestamp. This means the token becomes invalid immediately if the user changes their password or if a sufficient time has passed.

- Email Delivery: The system encodes the user's primary key (`uid`) and the token into a unique URL (e.g., `soulcare.com/reset/<uid>/<token>`) and emails it to the registered address.

- Validation: The reset view (`PasswordResetConfirmView`) decodes the UID, fetches the user, and validates the token against the current state of the user record before allowing the password update.

Code Snippet: Password Reset Confirmation Logic,

```
class PasswordResetConfirmView(APIView):
    def post(self, request):
        # ... decode uid and get token ...
        if not default_token_generator.check_token(user, token):
            return Response({"error": "Invalid or expired token"}, status=400)

        # If valid, set the new password (hashes it automatically)
        user.set_password(request.data['new_password'])
        user.save()
        return Response({"message": "Password reset successfully"})
```

Results & Security Validation:

- Session Persistence: The implementation of refresh tokens allows users to maintain their session across page reloads and browser restarts, significantly improving usability compared to simple session cookies.

- Recovery Reliability: The password reset email is consistently delivered, and the token validation prevents replay attacks (reusing a link after the password has been reset).

3.2 Administrative Governance Dashboard

Implementation: I architected a centralized command center to empower administrators with complete oversight of the platform's operations, ensuring compliance, safety, and quality.

- **User Management & Verification:** I developed a comprehensive user management module.
    - **Reusable Interface:** I built a modular `AdminUserTable` component in React that dynamically renders user lists based on the selected role (Doctor, Counselor, Patient). This component integrates sorting, searching, and pagination.
    - **Verification Logic:** To maintain a network of qualified professionals, I implemented a document based verification workflow. The system securely stores license documents uploaded during registration. In the admin dashboard, I added logic to fetch and display these documents via a secure URL (`license_document_url`). Admins can review the document in a new tab and then use a toggle switch to update the user's `is_verified` status in the database, which instantly grants the provider access to clinical tools.
- **Content Moderation System:** I engineered robust moderation pipelines for user generated content to maintain platform standards.
    - **Blog Management:** I implemented a full lifecycle management system for blogs. Posts submitted by providers are assigned a default status of "Pending". I built an admin interface (`ManageBlogsPage.tsx`) that filters posts by status. Admins can review content, edit metadata if necessary, and use custom API actions to transition posts to "Published" (visible to all) or "Rejected".
    - **Feedback Management:** I developed a testimonial moderation system (`ManageFeedbackPage.tsx`). User feedback is stored with an `is_approved=False` flag by default. I created a dedicated admin view that lists pending feedback with user details and ratings. Admins can "Approve" feedback, which triggers a backend update to `is_approved=True`, automatically publishing the testimonial to the public landing page via a filtered API endpoint.

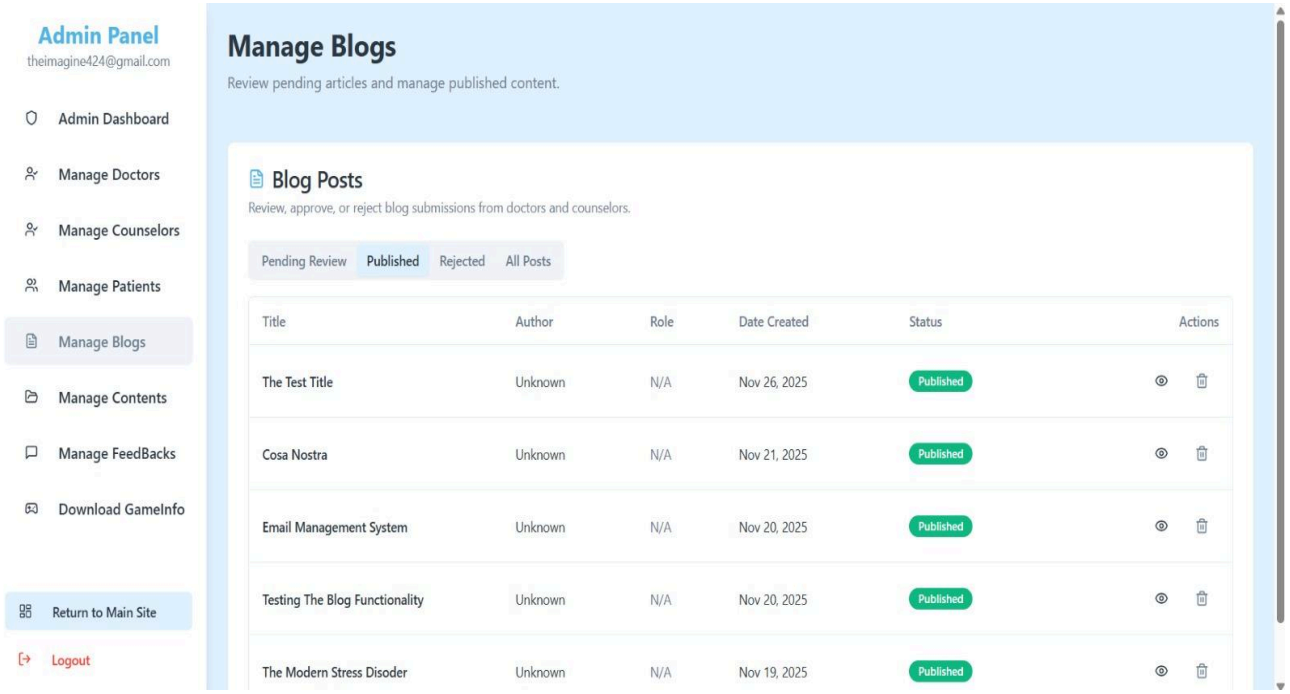
- **Data Analytics (Game Matrix):** To facilitate research, I implemented a data export utility. I wrote a custom Django view (`AdminGameDataExportView`) that accepts a `game_type` parameter. It queries the specific game result model, serializes the data, and uses Python's `csv` library to generate a file in memory, streaming it directly to the admin's browser as a downloadable report.

Results:

- **Governance Efficiency:** The centralized dashboard has significantly reduced the time required for administrative tasks. User verification now takes seconds, and the moderation queues ensure that no unapproved content reaches the public, maintaining a high standard of trust and professionalism.

- **Data Integrity:** The export feature provides researchers and admins with reliable, raw datasets for analysis without requiring direct database access

Figure 01: Blog Management Admin Dashboard



3.3 Clinical Management System

Implementation: The core of the clinical management functionality is the Appointment Scheduling and Management System, which I engineered to facilitate the complex interaction between patients and providers. My implementation focused on ensuring data integrity during the booking process and managing the appointment lifecycle through a strict state machine.

- **Appointment Booking Engine:** I developed a conflict resolution algorithm within the backend `AppointmentViewSet`. When a patient submits a booking request, the system does not merely create a record; it performs a real time validation check. It queries the database for existing appointments associated with the specific provider at the requested date and time. This server side validation effectively prevents double booking, ensuring that two patients cannot reserve the same slot simultaneously, even if they attempt to book at the exact same moment. To guarantee temporal accuracy across different time zones, I overrode the `save()` method on the `Appointment` model to calculate and store precise `start_time` timestamps by combining date and time inputs before persistence.

  Code Snippet: Appointment State Management,

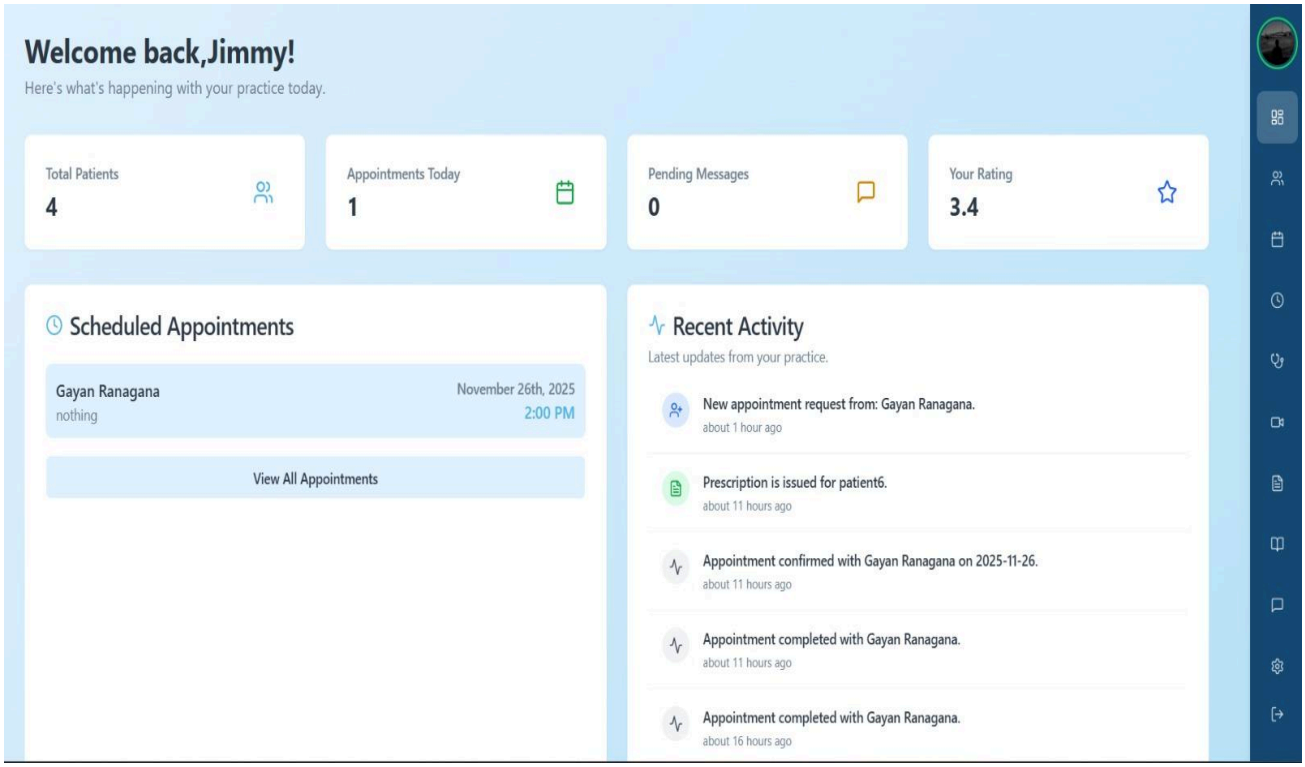
```
@action(detail=True, methods=['post'], url_path='update-status')
def update_status(self, request, pk=None):
    appointment = self.get_object()
    new_status = request.data.get('status')
    # Strict Permission Check
    if appointment.provider != request.user:
        return Response({'error': 'Permission denied'}, status=403)
    # State Transition Logic
    if new_status == 'completed' and appointment.status != 'scheduled':
        return Response({'error': 'Invalid transition'}, status=400)
    appointment.status = new_status
    # Audit Trail: Track who cancelled
```

```
        if new_status == 'cancelled':
            appointment.cancelled_by = 'provider'
        appointment.save()
        return Response(AppointmentReadSerializer(appointment).data)
```

- **Lifecycle State Machine:** I implemented a state management system to govern the appointment flow. Appointments transition through specific states: Pending (initial request) to Scheduled (confirmed by provider) to Completed (session finished) or Cancelled. I created custom API actions (e.g., update_status) to handle these transitions. These endpoints enforce strict business rules, such as preventing a completed appointment from being cancelled and ensuring only the assigned provider can confirm a request.

- **Provider Dashboards & Analytics:** To provide doctors and counselors with actionable insights, I built dashboard widgets that display real time practice statistics. I utilized Django's aggregation functions (Count, Avg) to calculate metrics like "Total Patients," "Appointments Today," and "Pending Actions" dynamically. This eliminates the need for providers to manually tally their daily tasks.

Figure 02: Doctor Dashboard



Results:

- **Streamlined Workflow:** The implementation successfully automated the scheduling workflow. Providers reported a significant reduction in administrative overhead as manual coordination via phone or email was eliminated. The system automatically routes requests and updates statuses, allowing clinicians to focus on patient care.

- **System Reliability:** The conflict resolution logic proved highly effective during testing. Concurrent booking attempts were correctly handled, with the system rejecting overlapping requests and maintaining schedule integrity. The timestamp precision ensured that appointments appeared correctly across different time zones without synchronization errors.

- **Data-Driven Insights:** The dashboard statistics provided immediate value, giving providers a clear, high level view of their practice's performance and daily workload upon login. The state machine ensured that the "Recent Activity" feeds remained accurate, providing a transparent audit trail of all interactions.

3.4 Communication and Engagement Modules

To facilitate continuous care and bridge the gap between scheduled sessions, I implemented a robust communication infrastructure. This module comprises a real time chat system for direct engagement and an automated email management system for reliable notifications.

Implementation:

1. **Real-Time Chat Infrastructure:** Moving beyond traditional HTTP request response cycles, I architected a full duplex communication system using WebSockets.
    - Technology Stack: I integrated Django Channels into the backend to handle asynchronous WebSocket connections. Redis was deployed as the channel layer backing store, serving as a high performance message broker to manage group communication.

- **Connection Lifecycle:** The frontend initiates a persistent WebSocket connection to a specific endpoint (e.g., `ws://chat/<room_id>/`). On the backend, a custom `ChatConsumer` intercepts this connection request. Crucially, it authenticates the user via the JWT token provided in the query string, ensuring that only authorized participants can join a chat room.

- **Message Handling:** When a user sends a message, the `receive` method in the consumer captures the JSON payload. It performs two critical actions:

    1. **Persistence:** Saves the message to the relational database (MySQL) to maintain a permanent chat history.

    2. **Broadcasting:** Immediately "broadcasts" the message payload to the Redis group associated with that chat room. This ensures that the recipient receives the message instantly without needing to refresh their page.

Code Snippet: Chat Consumer Implementation, This snippet demonstrates how the backend handles incoming WebSocket messages and broadcasts them to the group.

```
import json

from channels.generic.websocket import AsyncWebsocketConsumer

from channels.db import database_sync_to_async

class ChatConsumer(AsyncWebsocketConsumer):

    async def connect(self):

        self.room_name = self.scope['url_route']['kwargs']['room_name']

        self.room_group_name = f'chat_{self.room_name}'

        # Join room group

        await self.channel_layer.group_add(

            self.room_group_name,
```

```
            self.channel_name
        )
        await self.accept()
        async def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json['message']
        user_id = self.scope['user'].id
        # Save message to database (Async wrapper for DB call)
        await self.save_message(user_id, self.room_name, message)
        # Send message to room group (Broadcast)
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                'type': 'chat_message',
                'message': message,
                'sender_id': user_id
            }
        )
    async def chat_message(self, event):
        # Receive message from room group and send to WebSocket
        await self.send(text_data=json.dumps({
            'message': event['message'],
            'sender_id': event['sender_id']
})))
```

2. **Email Management System:** I engineered a centralized, transactional email service to keep users informed of critical updates. This system was designed to be modular and reusable across the application.

- **Configuration:** I configured Django's `EMAIL_BACKEND` settings to interface with an SMTP provider (for production) and the console backend (for development testing).

- **Utility Module:** Instead of scattering email logic across various views, I created a dedicated utility file (`utils.py`) containing specialized functions for each notification type (e.g., `send_welcome_email`, `send_appointment_confirmation`). This promotes code reuse and maintainability.

- **Dynamic Injection:** These functions accept context data (User objects, Appointment details) and dynamically inject them into the email body using Python f-strings.

- **Trigger Points:** Emails are triggered automatically at key lifecycle events. For instance, the `send_appointment_approved_email` function is called inside the `update_status` view immediately after a provider changes an appointment's status to "Scheduled".

  Code Snippet: Email Utility Function

```
from django.core.mail import send_mail

from django.conf import settings

def send_appointment_approved_email(appointment):
    """
    Sends a notification to the patient when their appointment is confirmed.
    """
    subject = "Appointment Confirmed - SoulCare"
    message = f"""
    Dear {appointment.patient.first_name},
    Great news! Dr. {appointment.provider.last_name} has confirmed your appointment
```

```
Date: {appointment.date}

Time: {appointment.time}

Please log in to your dashboard to join the video call at the scheduled time.

Best regards,

The SoulCare Team

"""

# Send email using Django's built-in wrapper

send_mail(

    subject,

    message,

    settings.DEFAULT_FROM_EMAIL,

    [appointment.patient.email],

    fail_silently=False,

)
```

3. **Content Library & Feedback:**

    - **Resource Sharing:** I developed a `Content` app that allows providers to upload therapeutic materials. The backend utilizes Django's `FileField` to manage file storage. I implemented custom permission logic (`get_queryset`) to ensure strict privacy. patients can only query content that has been explicitly shared with their user ID via a Many to Many relationship field (`shared_with`).

    - **Feedback Loop:** To foster quality assurance, I built a feedback submission system. The frontend collects star ratings and text reviews. These are submitted to a `Feedback` endpoint. I wrote backend logic to recalculate a provider's average rating every time a new review is approved by an admin, ensuring the public profile reflects up to date patient sentiment.

Results:

- **Instant Communication:** The WebSocket implementation successfully achieved low-latency messaging. During testing, messages sent between two active browser sessions appeared instantly, facilitating fluid conversation similar to standard messaging apps.

- **Reliable Notifications:** The Email Management System proved critical for user retention and adherence.

    - **Verification:** Registration emails with Links were delivered instantly, validating the user's identity.

    - **Adherence:** "Prescription Issued" emails prompted patients to log in and view their medication immediately, closing the loop on clinical care.

- **Patient Empowerment:** The Content Library and Feedback systems transformed the patient experience from passive to active. Patients successfully accessed shared resources for self care and provided valuable data on provider performance through the rating system.

4. CONCLUSION AND FUTURE WORK

4.1 Conclusion

The development of the SoulCare platform marks a significant step forward in linking professional mental healthcare with personal self care. This project successfully created a unified online system that solves the problem of fragmented mental health services. My personal work was essential in building a secure, scalable, and interactive foundation for this system.

By designing a custom Authentication and Security System with clear role based access, I ensured that private patient data remains safe. This setup allowed Patients, Doctors, Counselors, and Admins to access their own specific, easy to use dashboards within the same application. The Admin Dashboard introduced important quality controls. Features for verifying provider documents and checking user blogs and feedback help maintain the platform's professionalism and trustworthiness. Additionally, the Game Matrix Export feature demonstrates how the system supports research by allowing admins to download game data for studying patient performance.

For doctors and counselors, the Appointment Scheduling System and Digital Prescription Tool have simplified daily tasks significantly. By automating bookings, preventing double booking, and sending notifications automatically, the system saves providers valuable time, allowing them to focus more on patient care. Eliminating manual scheduling and paper records also reduces the chance of errors.

Finally, the addition of real time features like Chat and automatic Email Notifications has transformed the patient experience. These tools keep patients and providers connected between sessions. By offering instant support and timely reminders, SoulCare helps patients stick to their treatment plans and better manage their mental health.

In summary, the successful launch of these features proves that a modern web application can serve as a complete hub for mental health. SoulCare demonstrates how technology can be used to make healthcare more accessible, secure, and supportive for everyone.

4.2 Future Work

Although the current SoulCare platform is fully functional, there are several ways it can be improved in the future to better serve patients and providers.

- **AI Based Health Predictions:** We plan to add artificial intelligence (AI) to the system. By analyzing game scores and mood logs (while keeping user identities private), the system could identify patterns that suggest a patient's mental health is declining. This would allow doctors to intervene early and provide help before a crisis occurs.

- **Mobile Application:** Currently, SoulCare works on web browsers. Building a dedicated mobile app for iOS and Android would make the platform more accessible. Patients could easily track their mood, play games, and chat with their providers directly from their phones, anytime and anywhere.

- **Built-in Video Calls:** The system currently supports video consultations, but integrating a native video tool directly into the app would be better. This would mean users don't have to click external links to start a call, making the experience smoother and more secure.

- **Blockchain for Records:** To make medical records even more secure, we could use blockchain technology. This would create a permanent, unchangeable history of prescriptions and diagnoses, ensuring that patient data is transparent and cannot be tampered with.

- **Wearable Device Integration:** Connecting SoulCare with smartwatches and fitness trackers (like Fitbit or Apple Watch) would be a valuable addition. This would allow the platform to automatically track sleep patterns, heart rate, and activity levels, giving providers a more complete picture of a patient's physical and mental well-being.

- **Community Support Groups:** Adding a feature for safe, moderated community forums would allow patients to connect with others facing similar challenges. This peer support can be a powerful tool for recovery, reducing feelings of isolation.

- **Multilingual Support:** To reach a wider audience, the platform could be updated to support multiple languages. This would ensure that non English speakers can also access mental health resources and communicate effectively with providers.

5.REFERENCES

Django Documentation:

- Django Software Foundation. (2024). *Django Documentation (Version 5.2)*. Retrieved from https://docs.djangoproject.com/
- Django Software Foundation. (2024). *Django Authentication System*. Retrieved from https://docs.djangoproject.com/en/5.2/topics/auth/

Django REST Framework:

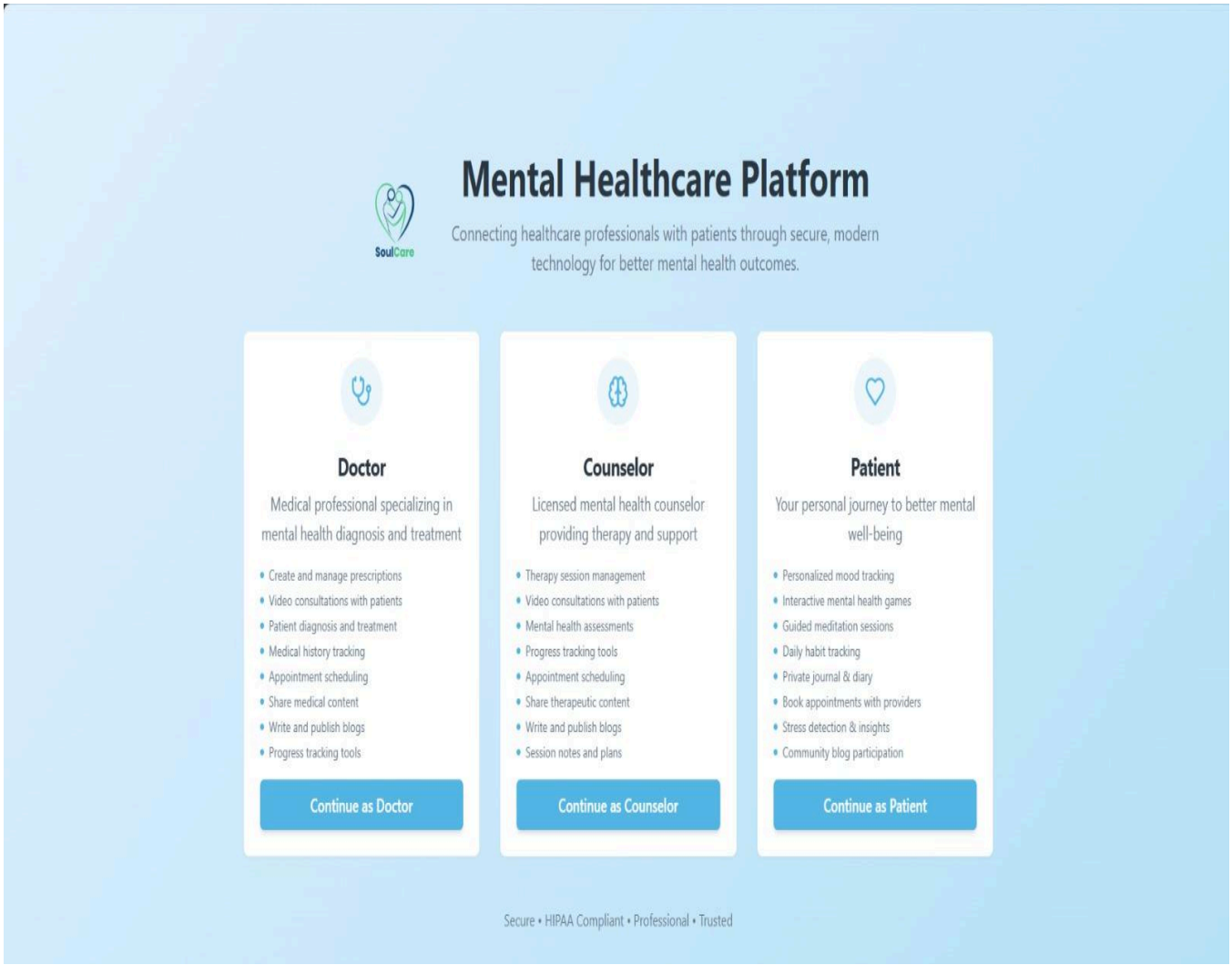
- Christie, T. (2024). *Django REST Framework Documentation*. Retrieved from https://www.django-rest-framework.org/

Frontend Development:

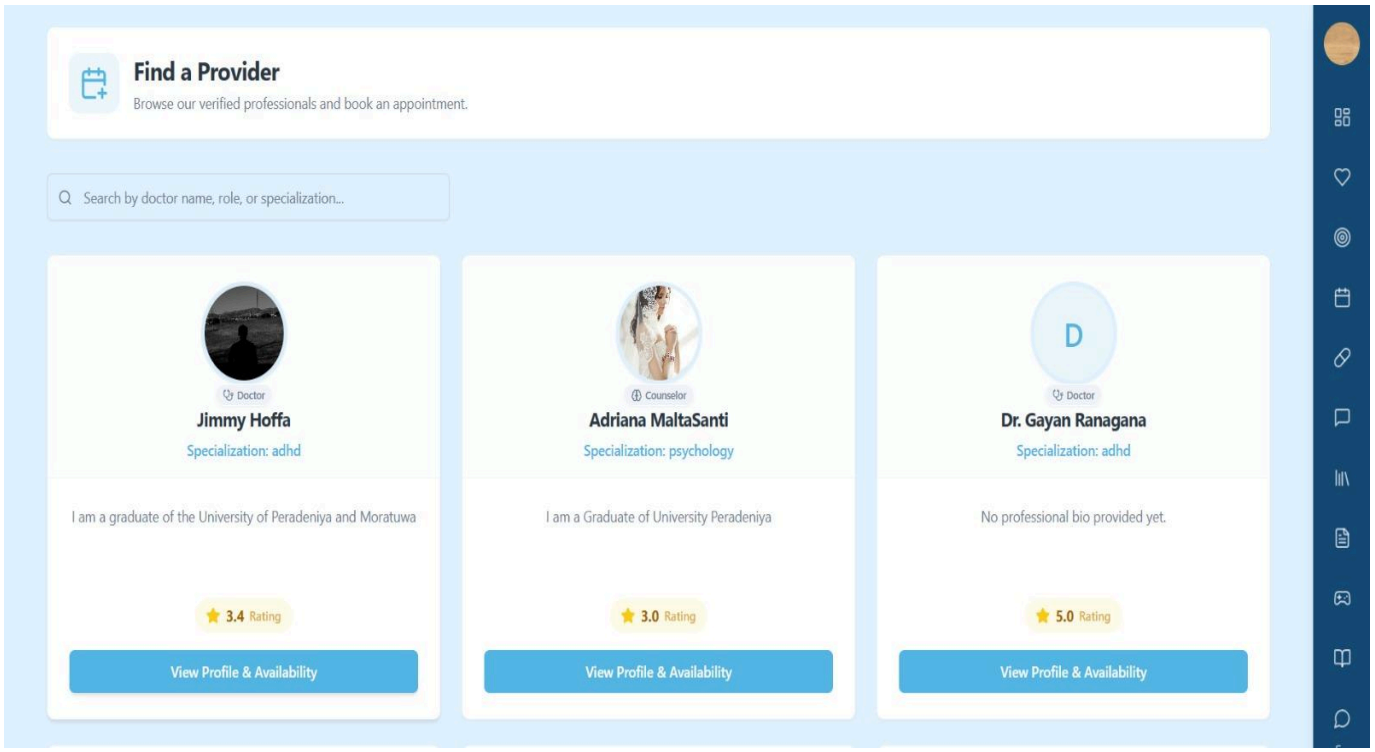
- React Documentation. (2024). *React: The Library for Web and Native User Interfaces*. Retrieved from https://react.dev/
- TanStack. (2024). *TanStack Query (React Query) Documentation*. Retrieved from https://tanstack.com/query/latest
- Tailwind Labs. (2024). *Tailwind CSS Documentation*. Retrieved from https://tailwindcss.com/docs
- shadcn/ui. (2024). *shadcn/ui Documentation*. Retrieved from https://ui.shadcn.com/

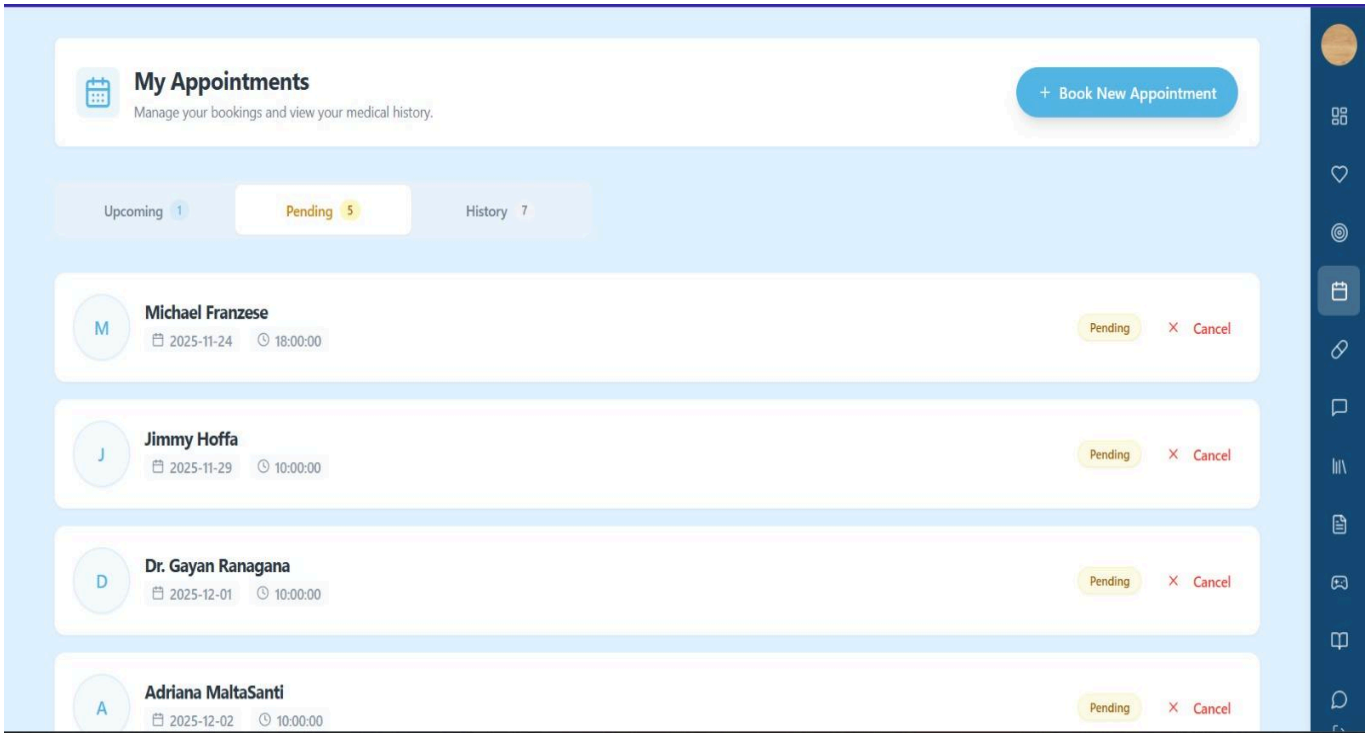
6. APPENDICES

Appendix A: Landing Page

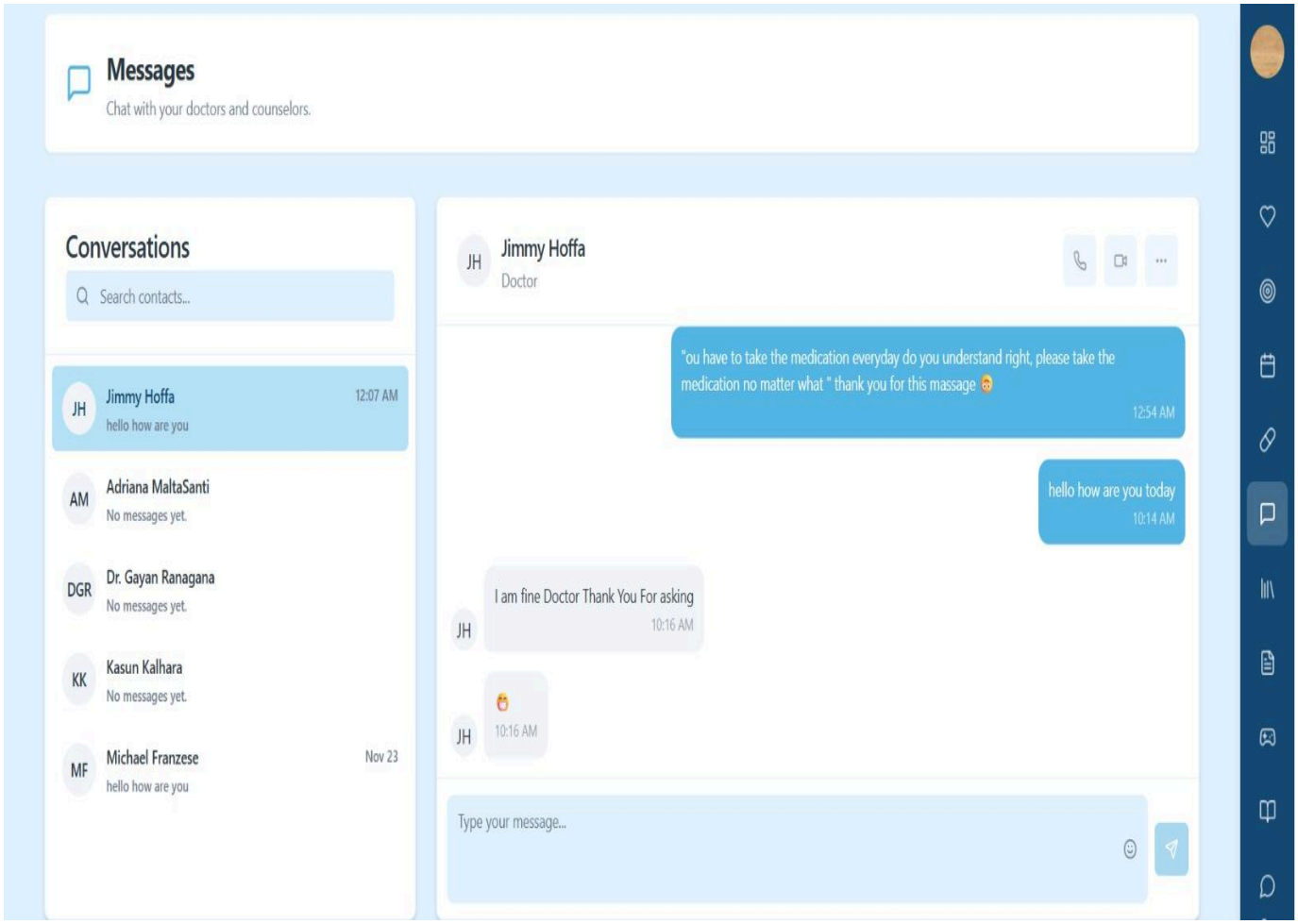


Appendix B: Find a Provider and My Appointments pages

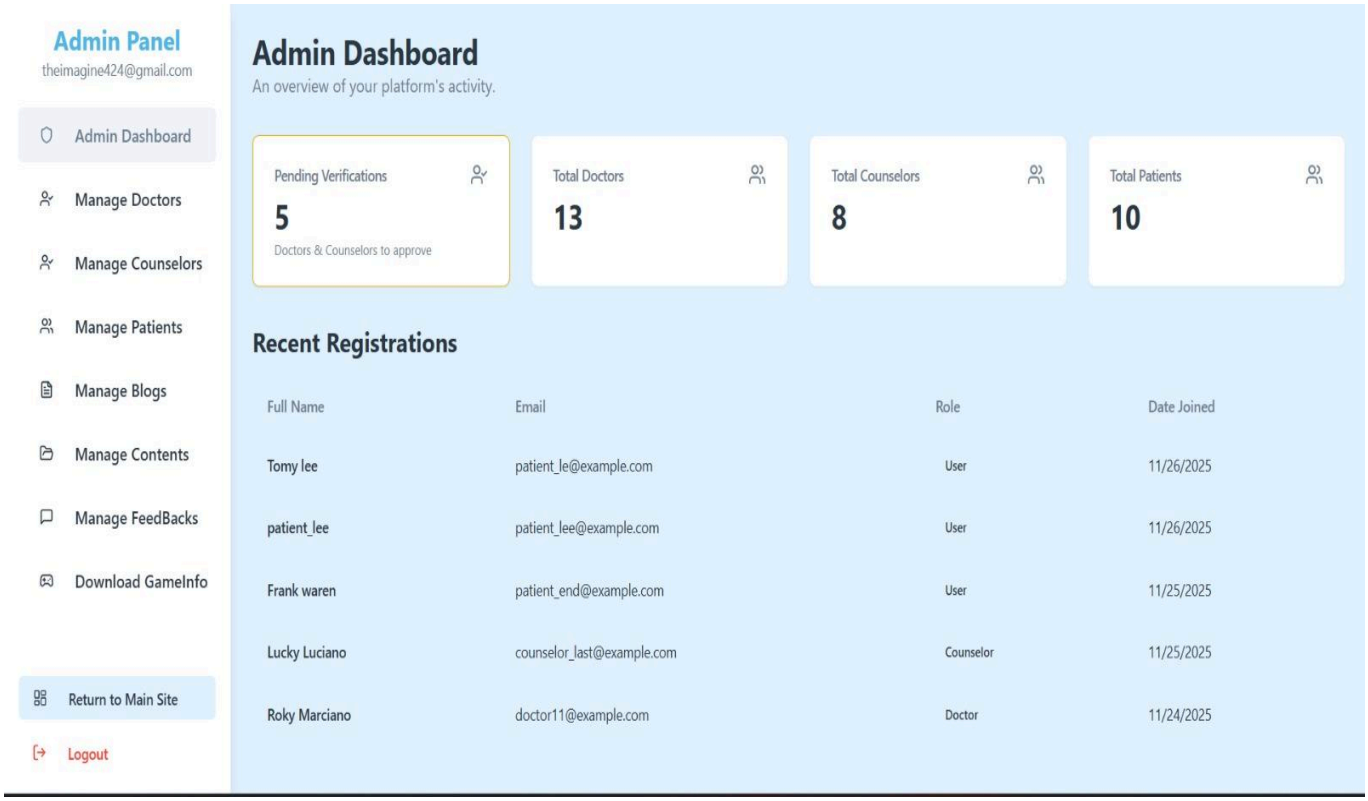


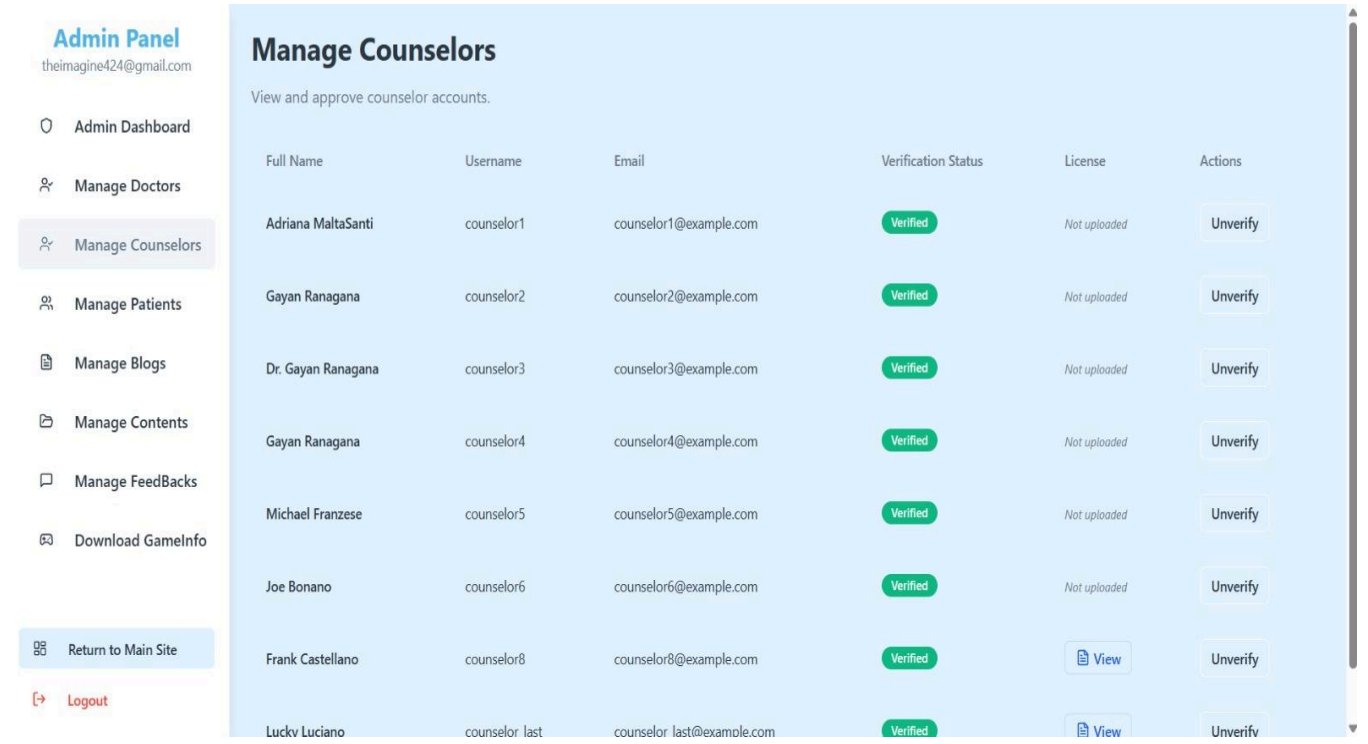


Appendix C: Messages Page



Appendix D: Admin Dashboard and Manage Counselors page





Appendix E: Content Library(Doctor and Counselor) and Manage Patients(Doctor and Counselor) pages

